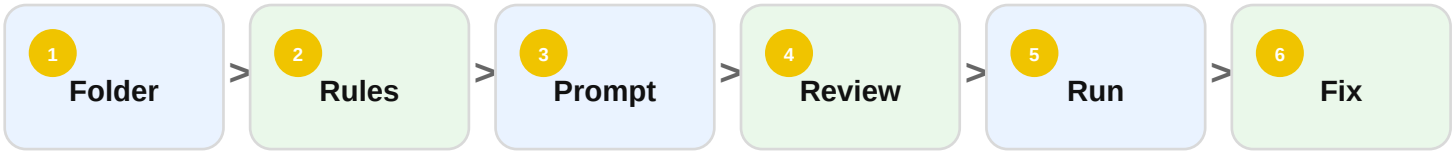
Coding Agents for Complete Beginners

A simple visual playbook for using Claude Code, Codex, and AI coding agents - even if you are not a coder yet.



Read this first

This guide is practical on purpose. It shows what to click, what to create, what to paste, and how to check the agent before trusting it.

What you will learn

- What a coding agent does and when to use it.
- How to create a safe project folder and instruction file.
- The exact prompts to plan, build, review, run, and debug.
- What to build first, and what to avoid until you are ready.

By AI with Mudit

1. What is a coding agent?

A coding agent is not magic. Think of it like a junior developer that can look inside your project and help change files.

1 Reads files

It can inspect your project files so it knows what already exists.

2 Makes a plan

It can suggest steps before editing, so you can correct the direction.

3 Edits code

After approval, it can create or update files for the task.

4 Runs commands

Some agents can help run tests, install packages, and start apps.

5 Explains changes

You can ask what changed and why it changed in simple words.

6 Still needs you

You approve, review, test, and decide. You are the boss.

Normal chatbot vs coding agent

Normal chatbot: gives code that you copy manually. Coding agent: can work inside the project folder and change files for you.

Important

Never blindly approve big changes. Ask for a plan, approve small edits, then test the result.

2. Claude Code vs Codex in plain English

Use the same thinking for both: clear goal, small task, review changes, test result.

Claude Code	Codex
Works like a terminal coding partner that reads your project and can help edit/run code.	AI coding agent from OpenAI for writing, reviewing, and shipping code.
Great when you want a project-folder workflow with commands and file edits.	Great when you want help planning, implementing, reviewing, and iterating code.
Use official Anthropic docs for current install/access steps.	Use official OpenAI docs/help for current access steps.

Do not memorize tool buttons

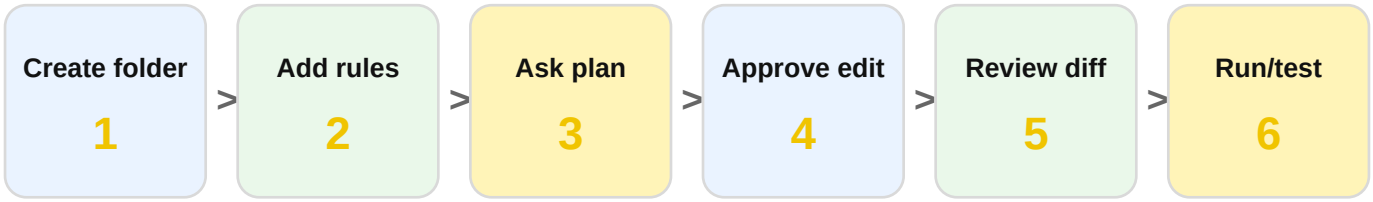
Interfaces change. The valuable skill is the workflow: give context, ask for a plan, approve small edits, review the diff, run the app, fix one bug at a time.

The mindset prompt

```
You are helping me as a careful junior developer.
Before editing, explain the plan in simple words.
Work in small steps.
After each edit, tell me what files changed and how to test it.
```

3. The whole workflow in one picture

This is the loop you should follow every time. Do not skip the review and test steps.



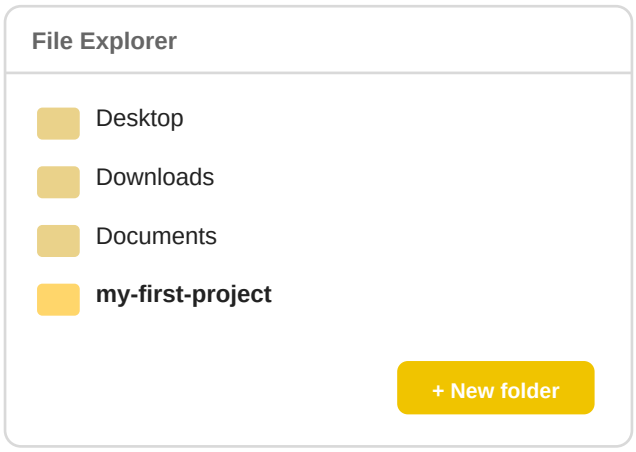
The rule

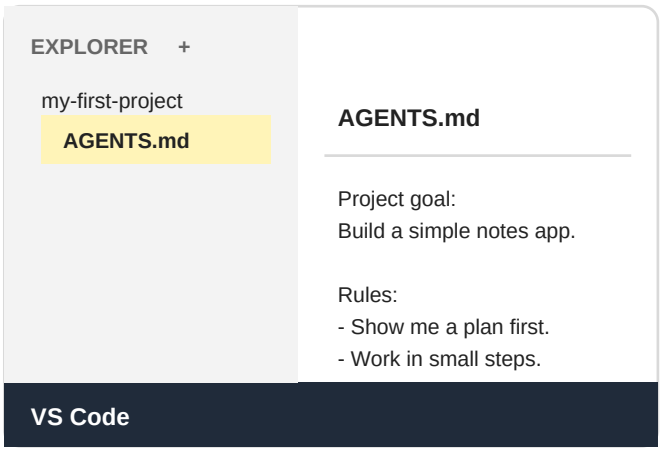
One clear task at a time. The smaller the task, the better the agent behaves and the easier it is to find bugs.

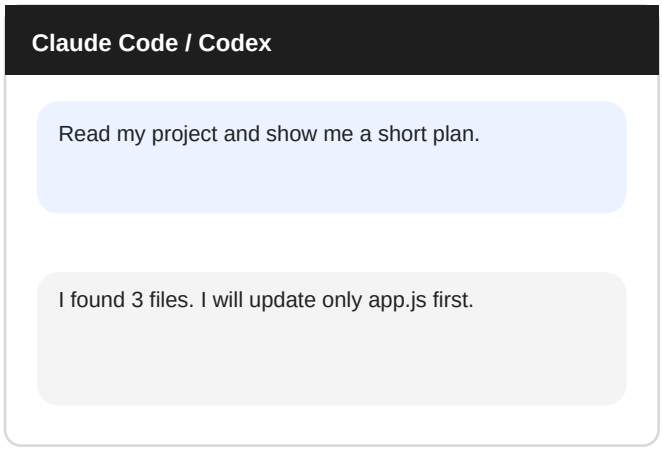
Bad prompt	Better prompt
Build a full startup app with login, payments, admin panel, AI, analytics, and database.	Build a simple notes app. First show me the plan. Do not edit files yet.
Fix everything and make it professional.	Fix only the save button bug. Explain the cause before editing.

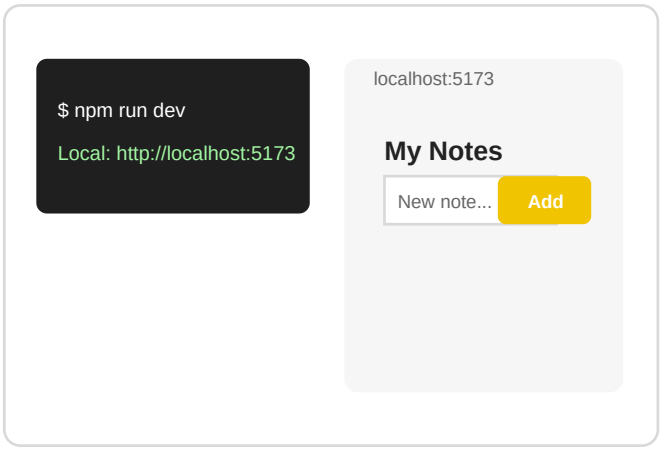
4. Know the 4 screens you will use

Beginners get confused because coding has many windows. This page shows what each window is for.







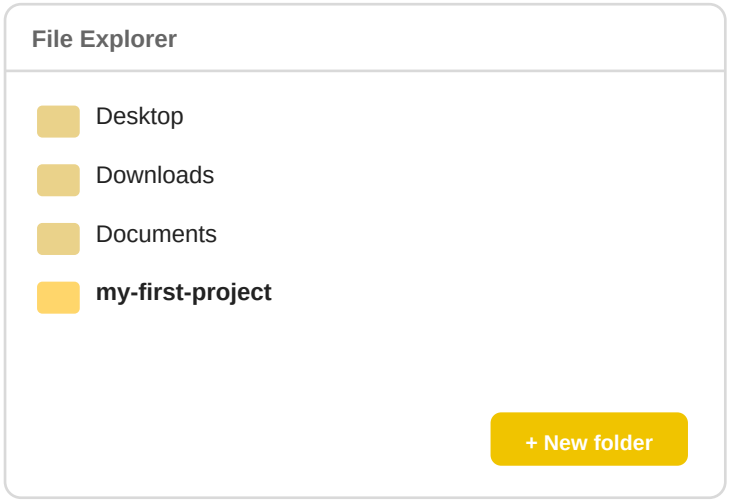


Simple meaning

Folder = where files live. VS Code = where files are visible. Agent chat = where you give tasks. Terminal/browser = where you run and test the app.

5. Step 1: Create a safe project folder

A project folder is one normal folder where all files for one app live.



Do this

Create one folder with a clear name like my-first-project, notes-app, or gym-landing-page. Keep everything for that project inside it.

Do not do this

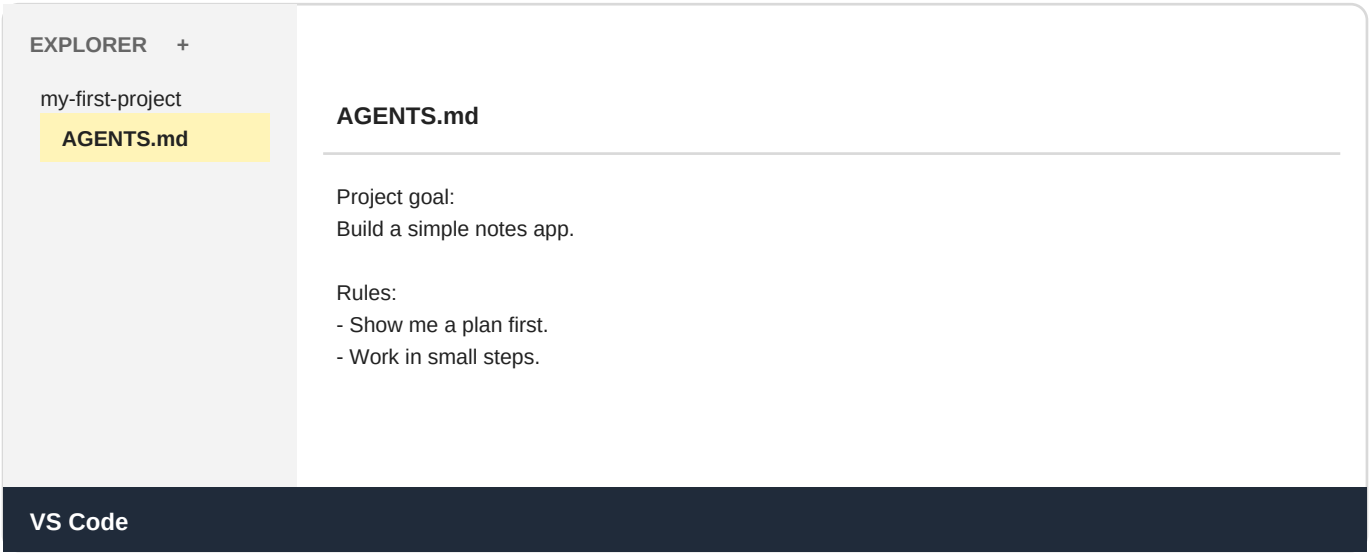
Do not keep app files scattered across Downloads, Desktop, WhatsApp, and random folders. Agents work better when the project is organized.

Beginner folder checklist

- One project = one folder.
- Folder name has no confusing spaces or symbols.
- You know where the folder is saved.
- You can open the same folder again tomorrow.
- Make a copy before big changes if the project matters.

6. Step 2: Create AGENTS.md or CLAUDE.md

This file is like a rulebook for the agent. It tells the agent how to behave inside this project.



Which file name?

Use AGENTS.md for agent instructions. If you are specifically using Claude Code, CLAUDE.md is also common. If unsure, use AGENTS.md.

Common mistake

Make sure the file is AGENTS.md, not AGENTS.md.txt. On Windows, turn on File name extensions if you cannot see the ending.

7. Paste these project instructions

Copy this into AGENTS.md or CLAUDE.md before asking the agent to build.

Copy-paste template

```
Project goal:
[write what the app should do]

My skill level:
Beginner. Explain changes simply.

Rules:
- Show me the plan before editing.
- Work in small steps.
- Do not delete unrelated files.
- After editing, tell me which files changed.
- Tell me how to run and test the app.
- Ask before risky commands.
```

Filled example

```
Project goal:
Build a simple notes app.

My skill level:
Beginner. Explain simply.

Rules:
- First give me a plan.
- Edit one feature at a time.
- Keep design mobile-friendly.
- Tell me files changed.
- Give a test checklist.
```

How to fill it

Replace only the project goal first. Keep the rules as they are. Later, when you become comfortable, add your own style preferences.

Why this is valuable

Without instructions, the agent guesses your working style. With instructions, it behaves more like a teammate who knows your rules.

8. Step 3: Ask for a plan first

Your first prompt should pause the agent before it edits files.

Claude Code / Codex

I want to build a simple notes app. First read the project folder and AGENTS.md. Then give me a short plan. Do not edit files yet.

Plan: 1) create index.html 2) add notes list 3) save notes in browser. Shall I start?

Copy-paste first prompt

```
I want to build a simple [type of app].
First read the project folder and my instruction file.
Then give me a short plan.
Do not edit any files yet.
```

What to look for in the plan

The plan should be short, clear, and safe. If it includes payments, login, database, scraping, or too many features, ask the agent to simplify.

9. The small-task method

Most beginner failures happen because the task is too big. Break everything into tiny pieces.

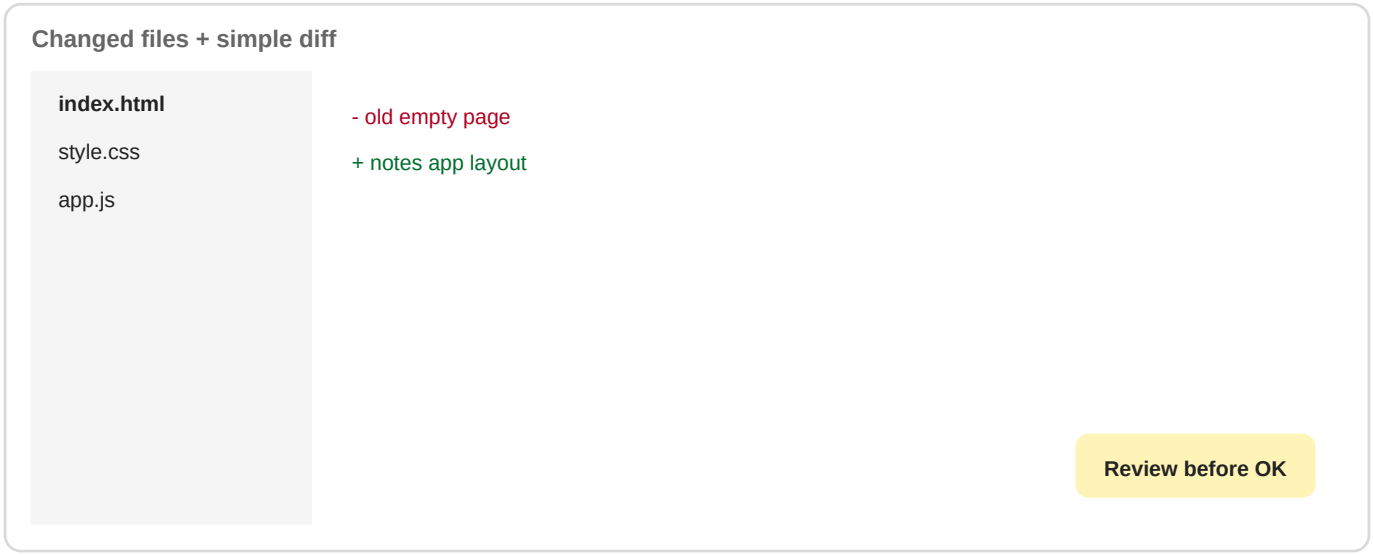
1. **Structure**
   Create only the basic HTML/layout.
2. **Style**
   Make it clean and mobile-friendly.
3. **Logic**
   Add one feature, like add note.
4. **Storage**
   Save data in browser/local storage.
5. **Polish**
   Improve empty states and error messages.

Prompt to keep it small

```
Good. Start with only [one small part].
Do not add advanced features yet.
After editing, tell me exactly which files changed.
```

10. Review changes before trusting them

You do not need to understand every line. First check whether the agent touched the right files.



5 checks before saying yes

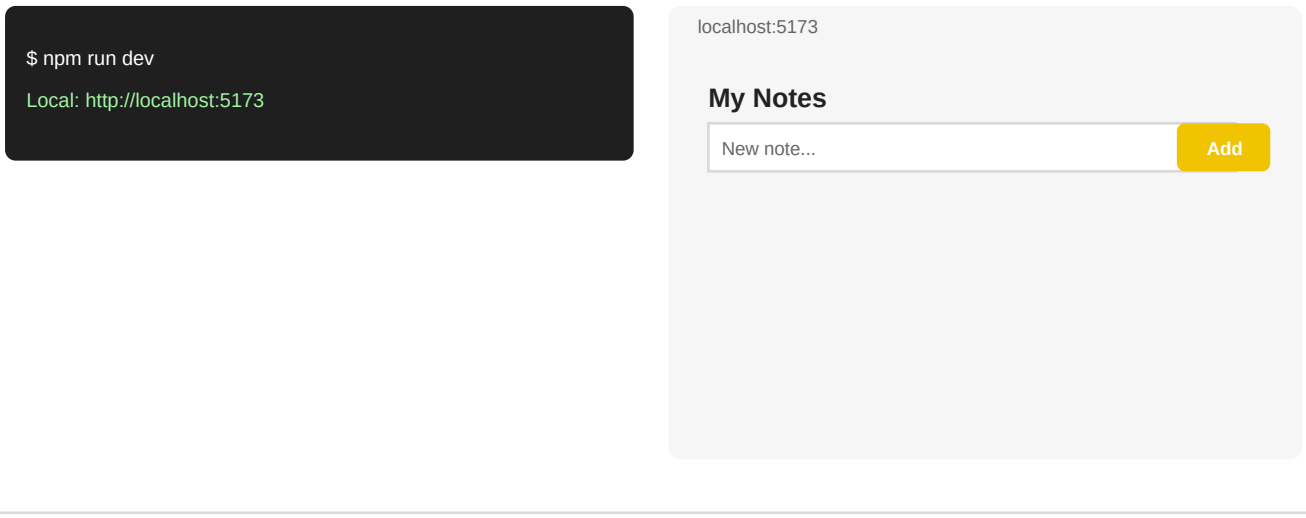
- Did it change only the files it said it would?
- Did it delete anything random?
- Does the app still open?
- Does the main feature work?
- Did the agent explain what changed in simple words?

Beginner shortcut

If you are confused, ask: Explain the changed files in simple words. What should I check manually?

11. Run and test the app

A coding agent is useful only if the app actually runs. Ask for the exact command and expected result.



Copy-paste run prompt

```
How do I run this project?
Give me the exact command.
Then tell me what I should see in the browser.
Also give me 5 simple things to test.
```

Test like a normal user

Click buttons. Type in fields. Refresh the page. Try empty inputs. Try long text. Try to break it before you trust it.

12. When something breaks

Broken code is normal. Good builders learn how to explain the bug clearly.

Bad bug report

It is not working. Fix it.

Good bug report

I clicked Add. Nothing happened. Console says: button is null.

Copy-paste debug prompt

```
Something is broken.
Here is what I did: [write what you clicked]
Here is what happened: [write the error or screenshot text]
First explain the likely reason simply.
Then fix only this one issue.
Do not rewrite the whole project.
```

Debug rule

Fix one bug at a time. If the agent changes 20 files for one small bug, stop and ask it to simplify.

13. Prompt library you will actually use

Replace the bracket text. Keep prompts short and direct.

Plan

Read the project and give me a short plan. Do not edit files yet.

Build

Implement only [small feature]. After editing, tell me which files changed.

Explain

Explain this code like I am new to coding. Keep it practical.

Review

Review your last changes. Mention any risk, bug, or file I should check.

Debug

Find the cause first. Fix only this one issue. Do not rewrite the project.

Improve UI

Improve the design only. Do not change the app logic unless needed.

Clean code

Clean the code without changing behavior. Tell me exactly what changed.

Next step

What is the safest next small step for this project?

14. First project: build a notes app

This is the best first project because it teaches layout, buttons, saving, and debugging without risky APIs.

1 Make page
Create index.html with title, input, Add button, notes list.

2 Style it
Make it clean, readable, and mobile-friendly.

3 Add notes
When I type and click Add, show the note below.

4 Delete notes
Add a delete button to each note.

5 Save notes
Use browser storage so notes stay after refresh.

6 Test it
Give me a checklist and fix one bug at a time.

Start prompt for notes app

```
Build a simple notes app in this folder.
First show me the plan. Do not edit files yet.
The app should let me add, delete, and save notes in the browser.
Keep the design simple and mobile-friendly.
```

15. Useful projects after your first app

Once you understand the loop, build tools that are useful in real life.

Gym landing page

One page for a gym: hero, plans, timings, WhatsApp CTA.

Resume website

Your intro, projects, skills, contact button.

Study tracker

Subjects, tasks, completed/not completed, local storage.

Content idea bank

Save reel ideas, hooks, captions, and status.

Invoice maker

Enter client details and generate a simple invoice preview.

Chatbot UI

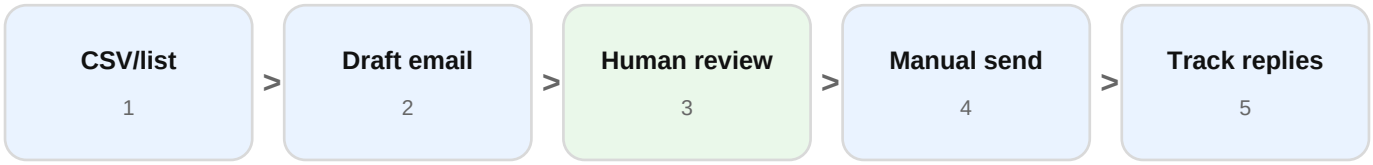
Frontend only first. Add API later when you understand keys.

Pick projects that teach the workflow

A boring finished app is better than an ambitious broken app. Start local. Add login, payments, APIs, and databases later.

16. How to use agents for real work safely

Your reel talked about outreach agents. This page shows the safe beginner version.



Safe outreach prompt

```
I have a list of businesses.
Do not send emails automatically.
Only draft 3 polite email examples.
Each email should be short, specific, and not spammy.
After drafting, explain why each email is relevant.
```

Do not start with auto-send

Beginners should create drafts first. Review every email yourself. Bulk sending without consent, targeting, and quality can damage trust.

17. Safety rules before using agents seriously

Agents move fast. That is useful, but it also means mistakes can become bigger faster.

1 **Secrets**

Never paste passwords, private keys, API keys, payment details, or private client data into prompts.

2 **Risky commands**

If you do not understand a command, ask the agent to explain it before approving.

3 **Backups**

Before a big change, copy the folder or use version control if you know Git.

4 **Permissions**

Do not let the agent delete files, send emails, or install random packages without explanation.

Responsibility reminder

The agent can move fast, but you are responsible for the result. Review before publishing, sending, or sharing anything.

18. 7-day roadmap

Do this for one week. The goal is not to become a full developer. The goal is to learn the workflow.

Day 1

Open the tool. Create one safe project folder.

Day 2

Create AGENTS.md / CLAUDE.md and paste rules.

Day 3

Build a tiny notes or to-do app.

Day 4

Ask the agent to improve the design only.

Day 5

Break one small thing and ask it to debug.

Day 6

Build one useful personal tool.

Day 7

Write what you learned and repeat with a new project.

Week goal

Learn how to think clearly, give small tasks, review changes, and test output. That skill compounds.

19. One-page cheat sheet

Screenshot this page or keep it open while using a coding agent.

Setup

```
1. Create one project folder
2. Open it in VS Code
3. Create AGENTS.md / CLAUDE.md
4. Paste your project rules
```

First prompt

```
Read the project and my instruction file.
Give me a short plan.
Do not edit files yet.
```

Build prompt

```
Build only [one small feature].
Work in small steps.
After editing, tell me which files changed.
```

Debug prompt

```
This is broken: [explain problem].
Find the cause first.
Fix only this issue.
Do not rewrite the whole project.
```

If you feel lost

Ask the agent: What is the safest next small step? Explain it like I am new to coding. Do not edit files yet.

Final reminder

Clear goal. Small task. Review changes. Test result. Repeat. That is the whole skill.

20. Sources and final note

Tools change quickly, so this guide avoids fragile installation steps. Use official docs for the latest access/setup instructions.

Official docs checked

Claude Code: Anthropic describes it as an agentic coding tool that can read codebases, edit files, run commands, and integrate with development tools. Codex: OpenAI describes it as an AI agent that helps you write, review, and ship code.

Your next 30 minutes

1 **Create a folder**
Name it notes-app or todo-practice.

2 **Paste rules**
Create AGENTS.md or CLAUDE.md and paste the template.

3 **Ask for a plan**
Do not allow edits in the first prompt.

4 **Approve one edit**
Run it, test it, then continue.

One final warning

Do not paste secrets, do not approve commands you do not understand, and do not automate outreach or payments until you know exactly what is happening.

Made for builders learning AI workflows

AI with Mudit